SELECTED ENGINEERING WORK

Artem Serdiuk

Backend & Full-Stack Engineer

Backend systems, integrations and internal tools - built around reliable data and reviewable outcomes.

Three engineering case studies, followed by selected application, AI and automation work.

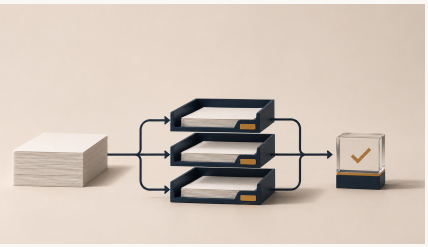
Cover images are conceptual illustrations. Architecture diagrams describe implementation; demonstrations are labelled.



01

Upwork workflow platform

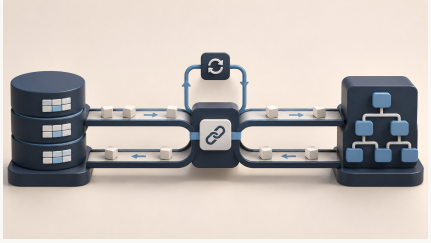
AI-assisted operations



02

Legal document workflow

Documents & Microsoft 365



03

PatientNow / GoHighLevel

Event-driven integrations

artom.serdyuk.600@gmail.com

Ukraine / open to remote work and European relocation

01 / AI-ASSISTED OPERATIONS

Upwork workflow platform

I built an internal workspace that brings job discovery, qualification, reusable project evidence, AI-assisted preparation and human review into one workflow. The difficult part was keeping the right account, source facts and reviewed draft together while external calls, background jobs and operator edits happened at different times.

Independent developer - backend, operator UI, integrations, testing and release tooling



THE PROBLEM

The initial requirement was an OAuth/MCP bridge for an existing automation workflow. The application expanded into a dedicated business tool with persistent workflow state, versioned rules, source-bound preparation and an operator workspace.

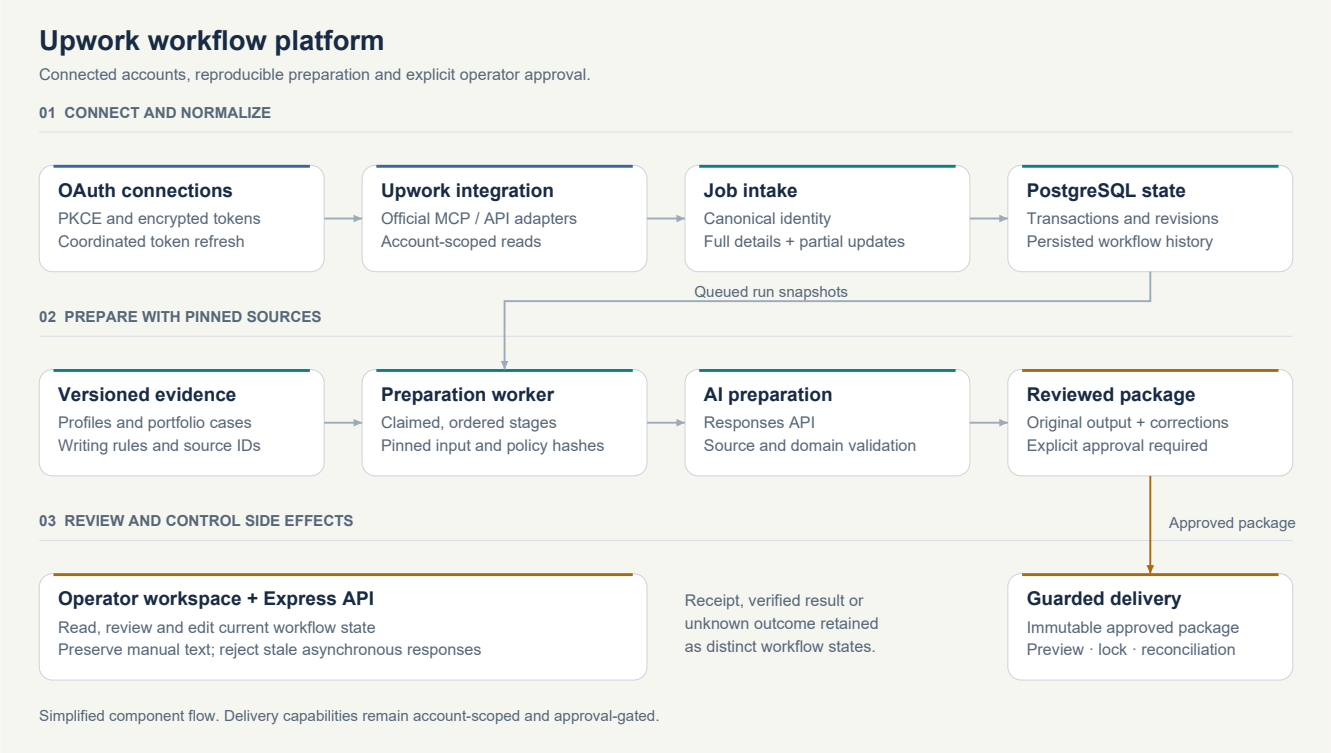
TECHNOLOGY

JavaScript ES modules / Node.js 20+ / Express5 / PostgreSQL/pg / Model Context Protocol SDK / OAuth2.1/PKCE / OpenAI Responses API / Zod/JSON Schema / HTML/CSS and vanilla JavaScript / Docker

The platform prepares and reviews work with guarded provider operations. A prepared package, acknowledged request and independently verified remote result remain separate states. Shared operator authentication and process-local coordination are current architectural constraints.

01 / SYSTEM DESIGN

Architecture & engineering decisions



Simplified implementation architecture. External systems and review boundaries are shown explicitly.

Bind authentication to the correct connection

Implemented OAuth discovery and PKCE, encrypted per-connection token storage, account-to-connection routing and single-flight refresh. Concurrent requests share the same refresh operation instead of racing a rotating refresh token.

Treat persistence uncertainty as a state, not a retry instruction

Used transactional PostgreSQL updates, a locked metadata row and revision-aware snapshots. If a COMMIT response becomes ambiguous, the store stops and requires reconciliation instead of silently falling back to old JSON or replaying a write.

Make long-running preparation reproducible

Added a durable run ledger with pinned input and policy hashes, version checks, ownership and ordered stages. Interrupted runs remain visible. A draft can be traced back to the specific job facts, profile, rule version and portfolio evidence used to create it.

Validate AI output against server-owned evidence

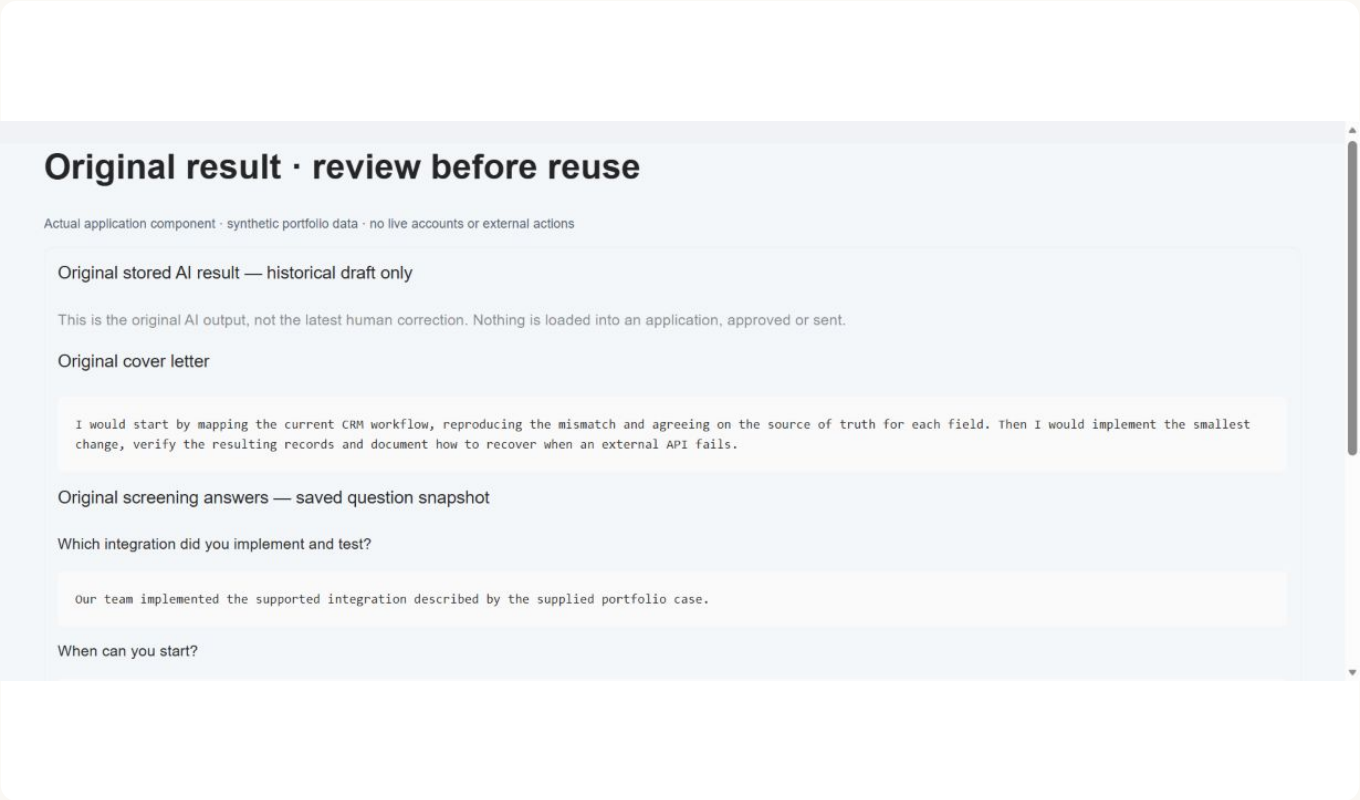
The model selects permitted evidence identifiers. The server resolves the original quotes and applies domain rules after schema validation. Unsupported experience, missing sources and unknown commitments remain blocked or explicitly unknown; human corrections are stored separately from original model output.

Application source is private. This public case describes a sanitized architecture.

Read the engineering case

01 / IMPLEMENTATION & VALIDATION

Implementation in context



Actual component renderer and styling, populated with synthetic portfolio data. Isolated demonstration; no live account, AI request or application submission.

Preserve richer records during partial refreshes

Separated complete job details from lower-confidence search observations. Equivalent money formats, reordered lists and contained description snippets do not replace authoritative fields. Meaningful contradictions flag a refresh while preserving the original draft and history.

Separate preparation, approval and remote delivery

Built immutable application packages, account-scoped previews, duplicate-send locks and delivery states that distinguish acknowledgement, verified delivery and unknown outcomes. The UI preserves unsaved manual text and rejects stale asynchronous responses.

5,388 passing tests

Saved regression run dated 6 November 2026. A separate, overlapping Linux run passed 1,987 tests with outbound networking disabled; the counts are not added.

A bounded performance experiment

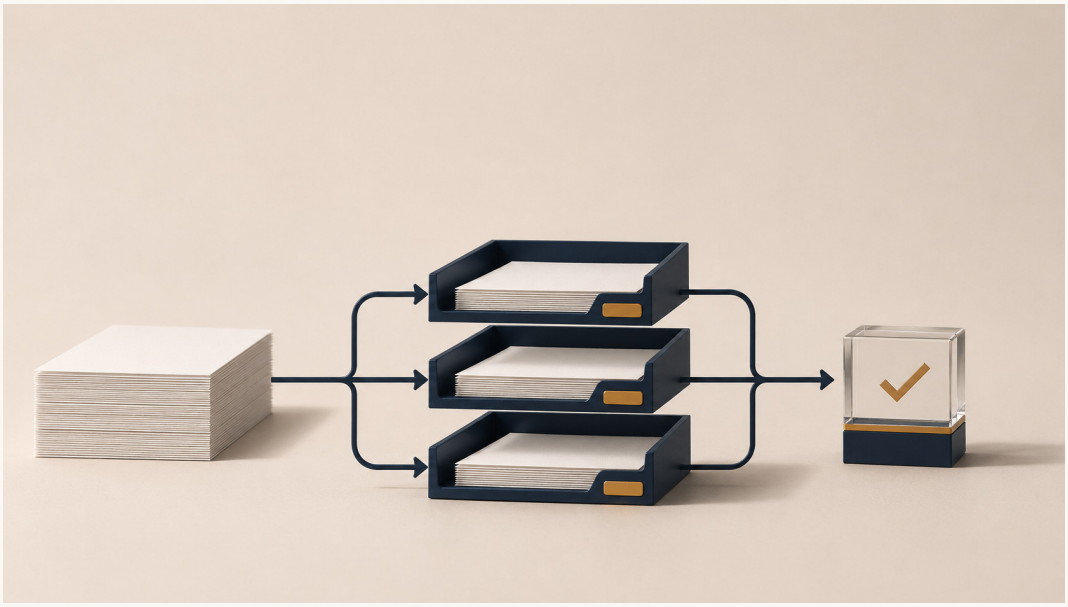
For one saved record, payload size changed from 545,987 to 71,343 bytes. Median read time changed from 2,300.4 to 1,325.7 ms across three GET requests before and after. This is a single-record comparison, not a production-wide benchmark.

02 / DOCUMENTS & MICROSOFT 365

Legal document workflow

I built a document operations platform that turns incoming emails and PDFs into isolated case drafts. It preserves reviewers' corrections, pins documents to reviewed versions and verifies external saved bundles before reporting preparation complete.

Sole full-stack engineer - application design, backend, admin interface, document pipeline, Microsoft integrations, browser preparation and Excel add-in.



THE PROBLEM

Legal intake was not a single PDF-to-form task. One email could contain several cases, scanned and fillable documents, incomplete fields and ambiguous attachments. Background extraction, human corrections and slow external systems had to coexist without mixing cases, discarding edits or repeating uncertain writes.

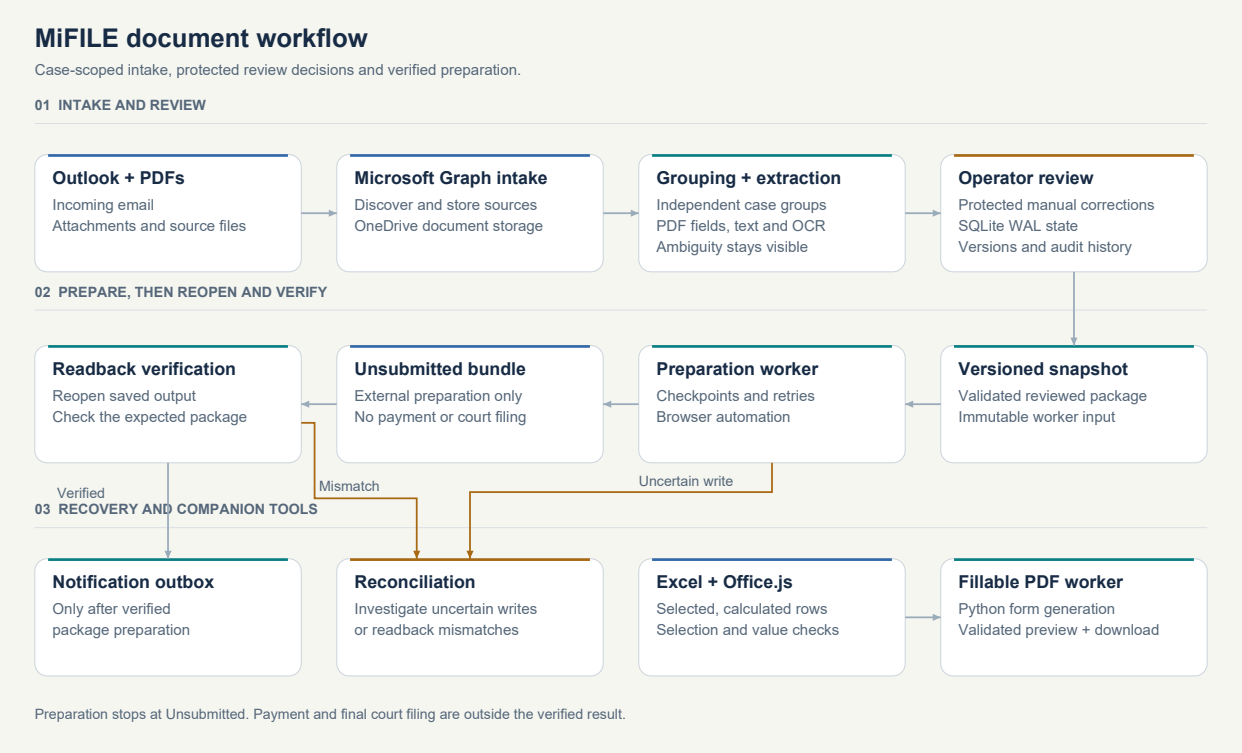
TECHNOLOGY

TypeScript / Node.js / SQLite WAL / Microsoft Graph / Azure Identity / OneDrive / Playwright / PDF.js / Tesseract OCR / Office.js / Python / pypdf / reportlab / Docker / HTML/CSS/JavaScript

The evidenced external workflow prepares and verifies Unsubmitted bundles. Completed payments and final court submissions are not claimed. Screenshots show the real application with synthetic QA data.

02 / SYSTEM DESIGN

Architecture & engineering decisions



Simplified implementation architecture. External systems and review boundaries are shown explicitly.

Treat each case as an independent unit

Several packages from one email retain separate source documents, review state, form choices, audit trails and job payloads. Ambiguous attachments stay visible in a review group. A failed attachment in one case does not block an otherwise complete sibling case.

Merge extraction around human corrections

Field-level protection preserves manually edited and intentionally blank values. Before an asynchronous result is applied, the worker rechecks source identity, replacement version and draft state. A late result cannot reopen or overwrite a changed package.

Pin the documents that were actually reviewed

Immutable SHA-256-addressed form files and source-content fingerprints connect a draft, its selected form version and the job snapshot. Replacing or changing a PDF invalidates earlier extraction/preparation assumptions.

Reconcile uncertain external writes

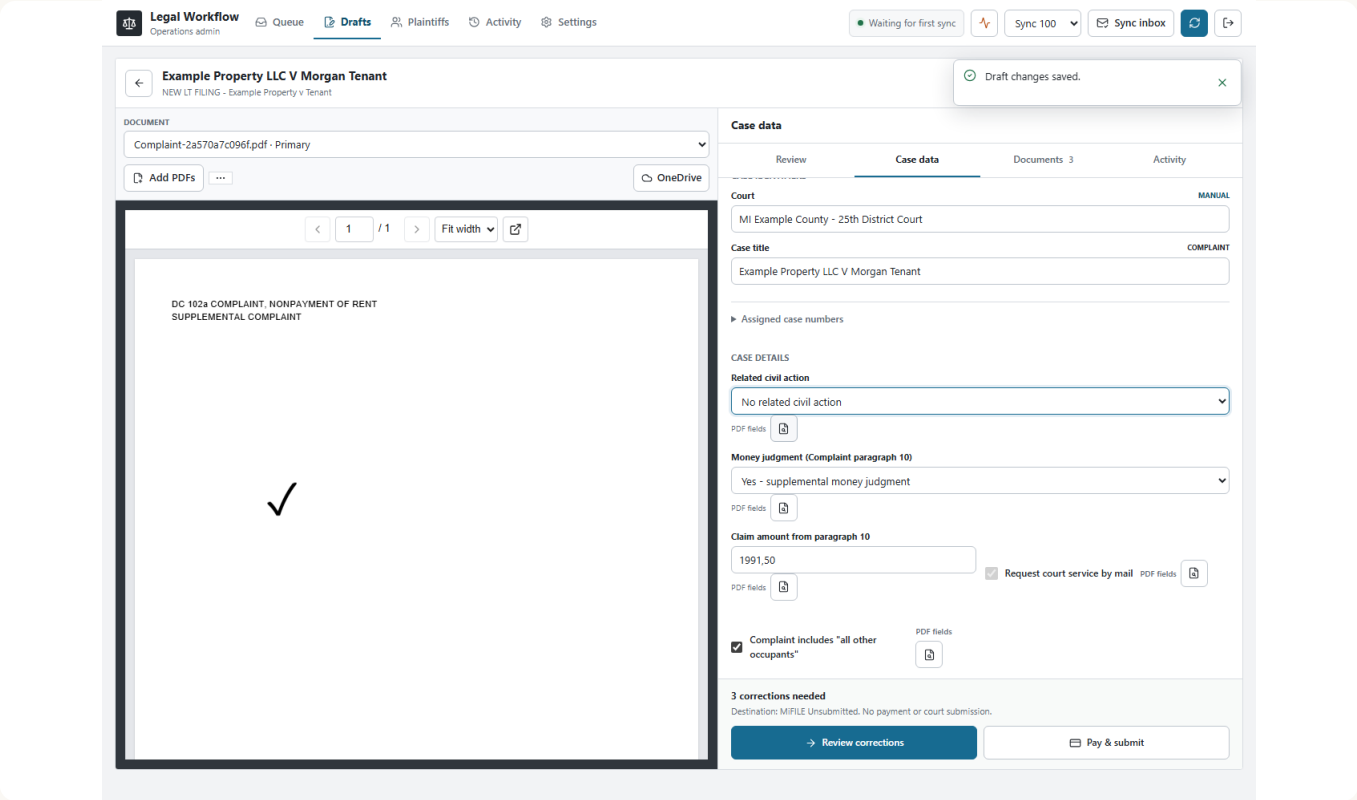
The worker persists checkpoints, verifies filing-type mappings and reopens the saved bundle to compare it with the source snapshot. If saving may have happened but cannot be confirmed, the workflow requires reconciliation rather than automatically creating another bundle. Graph read retries are bounded and explicitly separate from uncertain writes.

The public repository is a source snapshot. The current case also describes newer local implementation.

View the public source snapshot

02 / IMPLEMENTATION & VALIDATION

Implementation in context



Actual application UI with synthetic QA data: extracted fields retain visible source labels and manual review controls.

Generate PDFs from live Excel state

The Office.js task pane waits for calculation completion, reads selected rows and rechecks row/sheet identity after asynchronous work. A Python worker generates fillable PDFs and validates field values, widget appearances and text fit. Selection changes invalidate old output.

Keep email delivery independent and observable

A persistent notification outbox is tied to a verified prepared bundle. Duplicate or stale notifications are rejected; interrupted delivery becomes uncertain so the operator can check before resending. Workflow state, retry history and audits persist in SQLite WAL.

87 source PDFs

A saved offline multi-case acceptance report dated 6 November 2026 exercised 87 PDFs and recorded unchanged original source files. This is acceptance scope, not production throughput.

Editable PDFs from desktop Excel

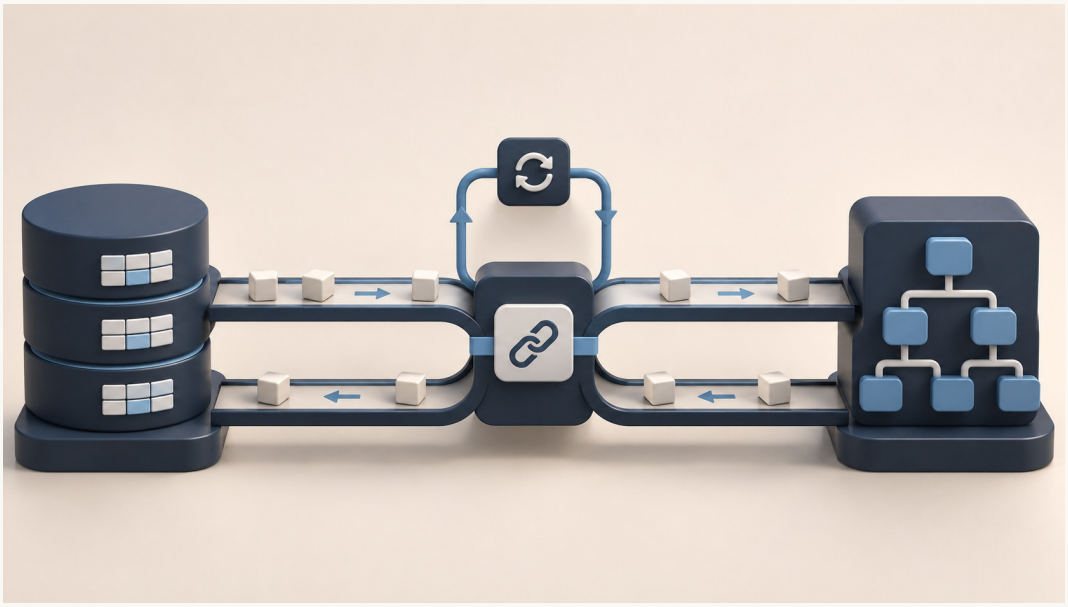
A saved Windows Excel acceptance record dated 29 October 2026 verified four generated PDFs, 126 compared field values and five rendered pages using fictional QA cases.

03 / EVENT-DRIVEN INTEGRATIONS

PatientNow / GoHighLevel

I built a TypeScript integration backend that keeps contacts, appointments, provider calendars and blocked time aligned across PatientNow and GoHighLevel. It also maps PatientNow orders and refunds into CRM opportunities. The engineering focus is predictable behaviour when events are duplicated, delayed, missing or interrupted by an external API failure.

Backend services, event normalization, identity resolution, synchronization rules, reconciliation, regression tests and deployment definitions. I also prepared an English operations and setup guide for handoff.



THE PROBLEM

Both platforms can change the same logical record, but they use different identifiers and expose different event and API semantics. A successful HTTP request is not enough: duplicate events can create duplicate records, a late update can undo a newer edit, and a missed reschedule can leave calendars inconsistent.

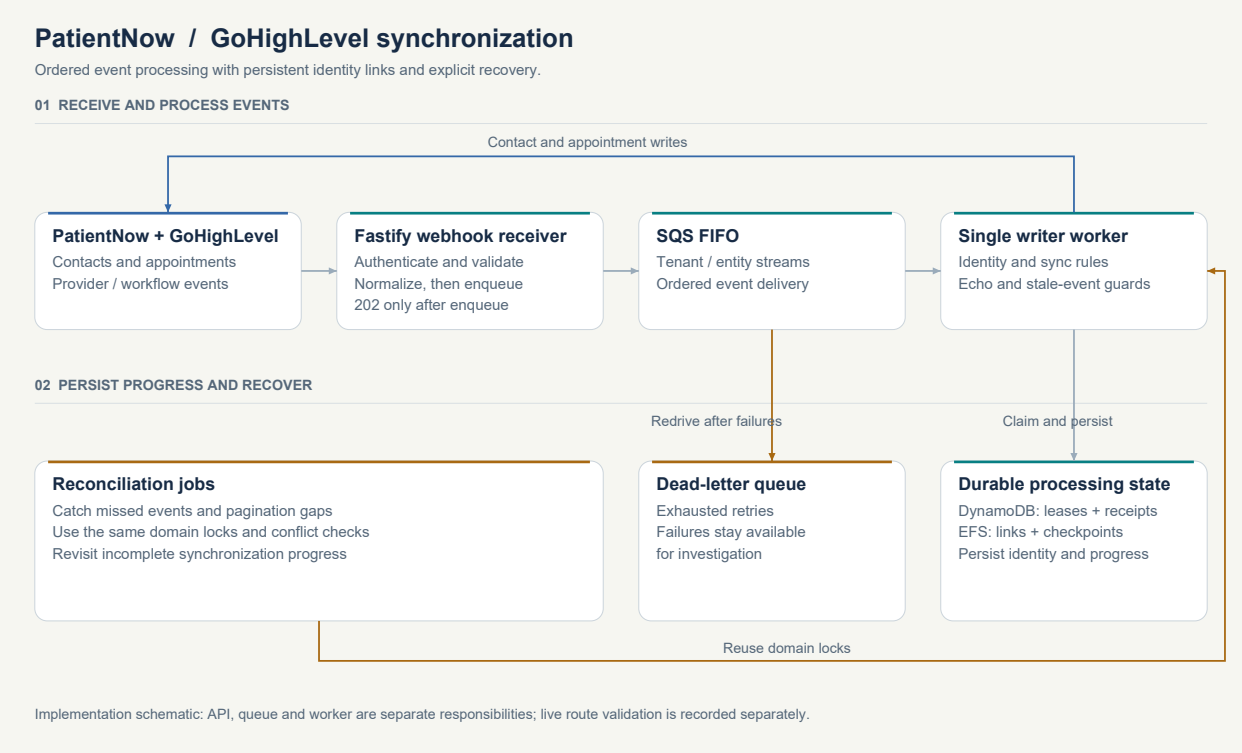
TECHNOLOGY

TypeScript / Node.js / Fastify / Axios / Zod / AWS SQS FIFO / DynamoDB / EFS / ECS/Fargate deployment definitions / Terraform / Docker

The current design deliberately uses one writer worker over EFS identity/history state. API replicas scale separately. A complete live cross-platform run, HIPAA certification, uptime and business-volume metrics are not claimed.

03 / SYSTEM DESIGN

Architecture & engineering decisions



Simplified implementation architecture. External systems and review boundaries are shown explicitly.

Order events before resolving identity

The two providers assign different record IDs. The queue therefore orders a tenant/entity-type stream instead of assuming that either provider ID is already a shared identity. The worker stops later messages in a failed group from overtaking the failure.

Give processing a durable owner

DynamoDB conditional writes acquire a processing lease and distinguish processed duplicates from work already in progress. Failed or expired work can be retried. SQS visibility is renewed during long operations, and a message is removed only after successful processing.

Refuse uncertain identity matches

Contact resolution checks persisted links, managed custom IDs and provider lookups. Ambiguous or conflicting matches fail visibly rather than choosing the first candidate. Pending-create state supports recovery when a provider may have created a record but its response did not contain an ID.

Treat time and cancellation as domain state

Appointment handling includes authoritative rereads, timezone normalization, echo suppression and cancellation tombstones. Moving an appointment uses create, verify and delete with a rollback path because the source API does not expose the needed update operation.

Current local implementation is broader than the earlier public source snapshot.

View the earlier public implementation

03 / IMPLEMENTATION & VALIDATION

Implementation in context

Operational scenarios

A webhook arrives twice

A durable processing lease distinguishes a processed duplicate from work that is still in progress. Message removal follows successful processing.

A create result is uncertain

Pending-create state supports recovery when the provider may have created a record but returned no identifier. Identity ambiguity remains visible.

An event never arrives

Paginated reconciliation compares linked records and checkpoints. Calendar and blocked-time updates do not depend entirely on webhook delivery.

Two sides change the same time block

Conflicting edits are surfaced for review. Appointment handling combines authoritative rereads, timezone normalization and cancellation tombstones.

Illustrative processing scenarios based on the implementation. This is not a live event log.

Recover from missing events

Paginated reconciliation compares linked records and checkpoints instead of assuming every webhook arrives. Provider-calendar and blocked-time reconciliation handles changes not represented by the selected webhook systems. Concurrent edits to a blocked-time record are surfaced as conflicts.

Represent sales without duplicating revenue

A PatientNow order maps to a CRM opportunity through a persisted order-ID link. Voids and full or partial refunds update the associated representation. Purchases flow from PatientNow to GHL; this is not a bidirectional payment-processing system.

74 offline tests

An isolated verification run dated 29 October 2026 passed 74 tests across 18 files with outbound networking blocked. This is historical regression evidence, not a fresh live end-to-end run.

24-page operations guide

Prepared an English deployment and handoff guide covering setup, configuration and operations.

APPLICATIONS & AI

Selected additional work

BACKEND LEAD / OCTOBER 2026-PRESENT

LeafSplit / AI model development

AI model development, training experiments and evaluation with the LeafSplit team. My work includes experimental tooling, automated checks and baseline comparisons, with an emphasis on reproducible results.

Python / NumPy / PyTorch experiments. Team contribution. Private model architecture and benchmark results are not disclosed.

TECHNICAL TAKE-HOME

Candidate Management Dashboard

A recruitment dashboard with candidate cards, status changes, filters and a skill catalog. Validated Express endpoints and Prisma transactions support optimistic React updates with rollback on failed requests.

React / TypeScript / Express / Prisma

Portfolio assessment; not a commercial production system.

INDEPENDENT APPLICATION

Shared wishlists & realtime groups

Shared wishlists, gift reservations, group contributions and realtime chat. The React/TypeScript application uses Supabase for authenticated flows, relational data, image storage and realtime subscriptions.

React / TypeScript / Vite / Tailwind CSS

Contribution tracking; no payment processing.

INDEPENDENT APPLICATION

CRM-KP sales workspace

A client dashboard and Kanban-style sales pipeline, with React/TypeScript state management, Express controllers, MongoDB models and Slack/email service modules.

React / TypeScript / Redux Toolkit / Material UI

Development snapshot; authentication integration needs further alignment.

EARLIER PAID FRONTEND WORK

HostShield frontend

Role-specific dashboards, document and review flows, account screens and a marketing website for a waiver-management product. The repository remains a historical frontend code example.

React / Next.js Pages Router / Redux / Tailwind CSS

The original product is closed.

INTEGRATIONS & CLIENT DELIVERY

Automation & interfaces

CLIENT WORK / ANONYMIZED

Shared lead intake & agent routing

A shared GoHighLevel intake flow across forms and landing pages. Field mappings, assigned-agent routing and notifications preserve application details when updating existing contacts.

GoHighLevel / forms / CRM workflows

CLIENT WORK / ANONYMIZED

Shopify storefront & funnel compatibility

Product and variant behaviour across storefront and funnel links. Focused changes preserve the expected cart, subscription and purchase-path behaviour of the existing integration.

Shopify / JavaScript / GoHighLevel

CLIENT WORK

Notion to GoHighLevel automation

A published Upwork portfolio example sending information from Notion to GoHighLevel through Make, adding low-code workflow automation alongside custom backend engineering.

Notion to GoHighLevel / Make

CLIENT WORK / ANONYMIZED

Payment-triggered course onboarding

A one-page course-sales funnel connects Stripe checkout to CRM updates, tags and access-email workflows through Zapier and webhooks, with responsive forms and follow-up sequences.

GoHighLevel / Stripe / Zapier / webhooks

CLIENT WORK / ANONYMIZED

White-label CRM customization

A customized GoHighLevel workspace with reusable JavaScript, CSS and responsive interface patterns. Work covers navigation, dashboards, smart lists, pipelines and role-aware visibility across sub-accounts.

Interface visibility is not server-enforced authorization.

CLIENT WORK / ANONYMIZED

Interactive mortgage intake

A Vue.js enquiry flow on AdsQuiz captures service intent, property details, indicative financing needs and timing before collecting contact information and consent. Includes integration-ready CRM delivery and segmentation workflows.

Vue.js / AdsQuiz / GoHighLevel

WORKING TOGETHER

Artem Serdiuk

Backend & Full-Stack Engineer

I build backend services and the tools people use to operate them: API integrations, data synchronization, document workflows and application interfaces.

ENGINEERING FOCUS

Data and integrations	Node.js / TypeScript / JavaScript / PostgreSQL / SQLite / REST APIs / OAuth / MCP
Workflows and interfaces	React / Next.js / custom operator UIs / Microsoft Graph / Office.js / GoHighLevel / Shopify
Documents and experiments	PDF extraction / OCR / editable PDF generation / Python / NumPy / PyTorch experiments

Software Engineering, Kherson National Technical University · expected graduation 2027

Remote from Ukraine; open to discussing relocation within Europe.

CONTACT & PUBLIC WORK

artom.serdyuk.600@gmail.com

GitHub
LinkedIn
Upwork
Portfolio repository